Evolutionary Characteristics

I	II	III	IV
Genesis	**Custom**	**Product (+rental)**	**Commodity (+utility)**
Uncharted			Industrialised
Chaotic			Ordered
Uncertain			Known
Unpredictable			Measured
Changing			Stable
Different			Standard
Exciting			Obvious
Future Worth			Low Margin
Unusual			Essential
Rare			Ubiquitous
Poorly Understood			Defined
Experimentation			Volume Operations
Differential			Operational Efficiency
Competitive Advantage			Cost of Doing Business

Types and stages of evolution

Stage of Evolution / Type	I	II	III	IV
Activities	Genesis	Custom	Product (+Rental Services)	Commodity (+Utility Services)
Practices	Novel	Emerging	Good	Best
Data	Unmodelled	Divergent	Convergent	Modelled
Knowledge	Concept	Hypothesis	Theory	Accepted

Characteristics of Activity stage

Stage of Evolution	I	II	III	IV
	Genesis	Custom	Product (+rental)	Commodity (+utility)
Characteristics				
Ubiquity	Rare	Slowly increasing consumption	Rapidly increasing consumption	Widespread and stabilising
Certainty	Poorly understood	Rapid increases in learning	Rapid increases in use / fit for purpose	Commonly understood (in terms of use)
Publication Types	Normally describe the wonder of the thing	Build / construct / awareness and learning	Maintenance / operations / installation / features	Focused on use
General Properties				
Market	Undefined market	Forming market	Growing market	Mature market
Knowledge management	Uncertain	Learning on use	Learning on operation	Known / accepted
Market perception	Chaotic (non-linear)	Domain of experts	Increasing expectations of use	Ordered (appearance of being linear) / trivial
User perception	Different / confusing / exciting / surprising	Leading edge / emerging	Common / disappointed if not used or available	Standard / expected
Perception in industry	Competitive advantage / unpredictable / unknown	Competitive advantage / ROI / case examples	Advantage through implementation / features	Cost of doing business / accepted
Focus of value	High future worth	Seeking profit / ROI?	High profitability	High volume / reducing margin
Understanding	Poorly understood / unpredictable	Increasing understanding / development of measures	Increasing education / constant refinement of needs / measures	Believed to be well defined / stable / measurable
Comparison	Constantly changing / a differential / unstable	Learning from others / testing the water / some evidential support	Feature difference	Essential / operational advantage
Failure	High / tolerated / assumed	Moderate / unsurprising but disappointed	Not tolerated, focus on constant improvement	Operational efficiency and surprised by failure
Market action	Gambling / driven by gut	Exploring a "found" value	Market analysis / listening to customers	Metric driven / build what is needed
Efficiency	Reducing the cost of change (experimentation)	Reducing cost of waste (Learning)	Reducing cost of waste (Learning)	Reducing cost of deviation (Volume)
Decision drivers	Heritage / culture	Analysis & synthesis	Analysis & synthesis	Previous experience

Doctrines

Communication	Be transparent (a bias towards open)	Focus on high situational awareness (understand what is being considered)	Use a common language (necessary for collaboration)	Challenge assumptions (speak up and question)
Development	Know your users (e.g. customers, shareholders, regulators, staff)	Focus on user needs	Think fast, inexpensive, restrained, and elegant (FIRE, formerly FIST)	Remove bias and duplication
	Use appropriate methods (e.g. agile vs lean vs six sigma)	Focus on the outcome not a contract (e.g. worth based development)	Be pragmatic (it doesn't matter if the cat is black or white so long as it catches mice)	Use standards where appropriate
	Use appropriate tools (e.g. mapping, financial models)			
Operation	Manage inertia (e.g. existing practices, political capital, previous investment)	Optimise flow (remove bottlenecks)	Think small (as in know the details)	Effectiveness over efficiency
	Do better with less (continual improvement)	Set exceptional standards (great is just not good enough)	Manage failure	
Structure	Provide purpose, mastery, & autonomy	Think small (as in teams, "two pizza")	Distribute power and decision making	Think aptitude and attitude
	Design for constant evolution	There is no one culture (e.g. pioneers, settlers and town planners)	Seek the best	
Learning	Use a systematic mechanism of learning (a bias towards data)	A bias towards action (learn by playing the game)	A bias towards the new (be curious, take appropriate risks)	Listen to your ecosystems (acts as future sensing engines)
Leading	Be the owner (take responsibility)	Move fast (an imperfect plan executed today is better than a perfect plan executed tomorrow)	Think big (inspire others, provide direction)	Strategy is iterative not linear (fast reactive cycles)
	Strategy is complex (there will be uncertainty)	Commit to the direction, be adaptive along the path (crossing the river by feeling the stones)	There is no core (everything is transient)	Be humble (listen, be selfless, have fortitude)
	Exploit the landscape			

Doctrines Reference

Communication

Be transparent (a bias towards open)

Share information openly and default to transparency rather than secrecy. This builds trust, enables better decision-making across teams, and prevents the organizational silos that slow strategy execution.

Examples

- Sharing real financial metrics and burn rates with the entire organization so all levels understand the business constraints and can optimize accordingly
- Publishing post-mortems with root causes and mistakes publicly—not just internally—to build credibility and learn from failures collectively

Focus on high situational awareness (understand what is being considered)

Ensure everyone understands the current context, market position, and strategic direction. Misalignment on what you're trying to compete on is the hidden killer of strategy execution.

Examples

- Using a shared Wardley map in all strategy discussions; teams see what's commodity (optimize cost), what's product (innovate), what's Genesis (experiment)
- Broadcasting the quarterly strategic intent and updating it monthly so teams understand why priorities shift, not just that they do

Use a common language (necessary for collaboration)

Establish shared terminology and frameworks across teams to reduce miscommunication and ensure clarity in discussions. Without this, teams optimize locally without understanding system-wide impact.

Examples

- Using Wardley Map positioning language (Genesis/Custom/Product/Commodity) across strategy discussions so business and engineering align on what phase a capability is in
- Agreed definitions of "MVP," "done," "technical debt," and "user need" to prevent engineering shipping completion while business meant learning

Challenge assumptions (speak up and question)

Encourage a culture where team members respectfully question decisions and assumptions rather than accepting them uncritically.

Examples

- Asking "why are we doing this?" before starting a new initiative
- Creating psychological safety where junior staff feel comfortable challenging senior leadership on plans

Development

Know your users (e.g. customers, shareholders, regulators, staff)

Understand who your actual and potential users are, including all stakeholders. Each has different needs; shipping for one user while ignoring another derails strategy.

Examples

- A fintech startup interviewing regulators and their legal team before designing the product, not after launch
- Mapping out all stakeholders (end-user, IT buyer, CFO, support team) and their conflicting needs to find the design that threads the needle

Focus on user needs

Prioritize solving real problems for users rather than building features you think they want. User needs should drive development priorities.

Examples

- Removing a fancy feature nobody uses in favor of improving the core workflow
- Listening to customer support tickets to identify unmet needs before investing in new features

Think fast, inexpensive, restrained, and elegant (FIRE, formerly FIST)

Design solutions that are rapid to implement, cost-effective, minimal in scope, and well-crafted. The elegance comes from removing everything non-essential, making speed possible without sacrificing quality.

Examples

- Testing market demand with a landing page + manual fulfillment (2 days, $100) rather than building the product first
- A team shipping a powerful API endpoint that handles 80% of use cases in 1 sprint instead of designing for all edge cases over 3 sprints

Remove bias and duplication

Eliminate redundant work and cognitive biases that lead to poor decisions. Two teams building identical solutions represents both wasted resources and missed signals about component maturity.

Examples

- Discovering three teams each built their own authentication system; consolidating to one shared service freed 6 months of capacity
- Challenging the assumption "we need custom code here" with data showing a commercial product meets needs at 10% of engineering time

Use appropriate methods (e.g. agile vs lean vs six sigma)

Select the methodology based on context: agile for uncertain/novel work, lean for optimization, six sigma for quality/stability. Using agile for a commodity operation is as wrong as using six sigma for Genesis exploration.

Examples

- Agile/experimentation for a new AI feature where no one knows what works; lean for supply chain optimization; six sigma for manufacturing
- Realizing you're in commodity phase and shifting from sprint planning to batch-based lean kanban

Focus on the outcome not a contract (e.g. worth based development)

Pay teams for achieving desired outcomes rather than completing specified tasks. This aligns incentives with actual value delivery and prevents the trap of shipping useless features on schedule.

Examples

- Contracting a vendor to reduce customer onboarding time by 50% (outcome), allowing them to choose the method, rather than demanding they "build module X"
- Funding a team based on "reduce support ticket volume by 30%" not "complete 40 story points," reversing perverse incentives

Be pragmatic (it doesn't matter if the cat is black or white so long as it catches mice)

Focus on what works in practice rather than adhering to ideology or "the right way." This doctrine is often in tension with "Set exceptional standards"—knowing which to prioritize requires strategy context.

Examples

- Using a legacy Java system to ship a critical market feature in weeks rather than rewriting in Go (the "right" language)
- Accepting manual operational processes temporarily to scale instead of waiting for perfect automation

Use standards where appropriate

Adopt industry standards to enable interoperability and reduce reinvention, but don't blindly follow standards that don't fit your context or phase. Standards matter most in commodity phases.

Examples

- Adopting REST and JSON as interfaces instead of proprietary formats—this reduces switching costs and attracts integrators
- Pushing back on a compliance standard that doesn't actually reduce risk, but meets industry expectation for your market

Use appropriate tools (e.g. mapping, financial models)

Leverage the right tools and techniques for analysis and planning. A Wardley map handles competitive uncertainty differently than a Gantt chart; a financial model handles scenarios differently than a business plan.

Examples

- Using Wardley mapping to identify which capabilities to build vs buy vs partner based on positioning and evolution, rather than a spreadsheet feature list
- Using scenario planning ("what if AI becomes a commodity in 18 months?") instead of linear forecasts to stress-test strategy

Operation

Manage inertia (e.g. existing practices, political capital, previous investment)

Recognize that established practices, relationships, and sunk costs create resistance to change. Strategy fails not because it's wrong, but because inertia is underestimated and unmanaged.

Examples

- A team invested 2 years building a custom database; migrating to a managed database isn't a technical problem, it's an inertia problem (pride, fear)
- A business leader genuinely committed to moving upmarket but has compensation tied to volume—the incentive structure perpetuates old strategy

Optimise flow (remove bottlenecks)

Identify and eliminate the constraints that slow down value delivery. Improving things outside the constraint is waste; focus ruthlessly on what actually blocks velocity.

Examples

- Discovering approval process (not code) is the constraint, reducing sign-off steps from 5 to 2, gaining weeks of time
- Automating the contracting/legal review (not the feature implementation) that's blocking deals from closing

Think small (as in know the details)

Understand the specifics and details of operations rather than operating only at abstract levels. Small details often contain important truths.

Examples

- A CEO personally observing the customer service process to understand frustration points
- A manager running an on-call shift to understand what actually breaks in production

Effectiveness over efficiency

Prioritize doing the right things (effectiveness) over doing things well (efficiency). You can be extremely efficient at the wrong work; that's just waste at scale.

Examples

- Stopping optimization of an on-premise product 6 months before switching to SaaS—you're being efficient at the wrong thing
- Realizing you're building features for the wrong buyer persona and pivoting, even if your feature factory is efficient

Do better with less (continual improvement)

Adopt kaizen: continuous, incremental improvement that compounds over time into extraordinary gains. This is about systems, not heroic effort.

Examples

- A manufacturing team reducing defects by 2% per quarter—in 18 months that compounds to 40% reduction without new capital
- Each sprint, the team investigates one thing slowing deployment; small changes to CI/CD stack up to 5x faster releases

Set exceptional standards (great is just not good enough)

Establish a culture where good enough is genuinely unacceptable. Pursue excellence and hold it as the baseline. This is in explicit tension with pragmatism—strategy determines which to emphasize.

Examples

- Shipping a feature 2 months late rather than with substandard UX because the product will live for years and poor design compounds
- Building blameless post-mortem culture not just to learn, but because accepting mediocre reliability corrodes teams over time

Manage failure

Expect failures, learn from them, and build systems that are resilient. Punishment for failures suppresses learning.

Examples

- Blameless post-mortems that analyze what happened without pointing fingers
- Designing systems with redundancy and graceful degradation to handle failures

Structure

Provide purpose, mastery, & autonomy

Give people meaning in their work (purpose), opportunities to get better at their craft (mastery), and freedom to decide how to do it (autonomy). Without all three, you lose good people and get surface compliance.

Examples

- Connecting daily sprint work to the company's mission and market impact, not just the feature list
- Engineers choosing their tech stack and approach within business constraints—building with agency, not just executing specs

Think small (as in teams, "two pizza")

Keep teams small enough that they can be fed with two pizzas (~8 people). Small teams make decisions faster, have better signal, and align more naturally. Beyond this, you need coordination overhead.

Examples

- Splitting a 25-person team into 3 autonomous squads; decision latency dropped and the budget debates stopped
- A team large enough to have status meetings about other status meetings is already too large

Distribute power and decision making

Push decisions down to people closest to information rather than centralizing all decisions at the top. Leaders set constraints/direction; teams choose methods within those bounds.

Examples

- Squad leads deciding their tech stack and deploy cadence within architectural guardrails, not waiting for central approval
- Customer support staff making refund decisions up to $10K without escalation—faster resolution, and they're closest to the truth about product failures

Think aptitude and attitude

Hire for attitude and learning ability; aptitude/skills can be trained over months. Attitude and culture fit are structural—they're almost impossible to change. Hiring for narrow skills and bad attitude is a trap.

Examples

- Hiring someone smart and curious who's never used your stack, investing 3 months to train them, versus hiring a narrow specialist with entitlement
- A hiring rubric that weights "evidence of learning from failure" over "previous title or certification"

Design for constant evolution

Build organizational structures, processes, and systems that can adapt as the world changes. Rigid structures become constraints; design them for reconfiguration.

Examples

- Team structures that reorganize quarterly without disruption because responsibilities are clearly defined
- Loosely coupled systems and architecture so you can move teams between capabilities without major integration work

There is no one culture (e.g. pioneers, settlers and town planners)

Organizations need different types of people and teams for different work. A Pioneer building Genesis products thinks differently from a Town Planner optimizing commodities. Value the difference; don't force all teams into one cultural mold.

Examples

- R&D teams (pioneers) given experimental freedom; ops teams (town planners) given process discipline; they need different incentives
- Sales to Enterprise (requires harmony, planning) operates differently from Sales to Startups (requires speed, adaptability); both needed, both different

Seek the best

Pursue excellence in hiring and development as a core strategic activity. Talented teams compound; average teams decline. This is a multiplier, not a nice-to-have.

Examples

- Spending 3 months recruiting one exceptional person (10x impact) instead of onboarding 5 adequate ones quickly
- Identifying high performers and creating visible pathways to leadership before competitors recruit them

Learning

Use a systematic mechanism of learning (a bias towards data)

Establish structured processes to gather and analyze data rather than relying on intuition or anecdote. Decisions should be evidence-based; debate with data, not opinion.

Examples

- Instrumentation in every user journey so you see where drop-off happens, not where you guess it does
- A/B testing features before full rollout; decisions made by effect size, not executive preference

A bias towards action (learn by playing the game)

Learn through doing rather than endless planning. Small experiments teach faster than elaborate prediction models. Ship to learn, don't plan to perfection.

Examples

- Launching a minimal version of a product with early customers in 4 weeks instead of spending 4 months perfecting it
- Running week-long operational experiments instead of debating for a month

A bias towards the new (be curious, take appropriate risks)

Encourage exploration and calculated risk-taking. Innovation requires willingness to try new approaches and fail. The risk isn't in experimentation; it's in not experimenting while competitors do.

Examples

- Allocating 10% time for engineers to explore emerging tech; some experiments fail, some become platform advantages
- A venture team with explicit permission to "fail fast on bets that could be 10x," not tinkering optimization

Listen to your ecosystems (acts as future sensing engines)

Pay close attention to partners, customers, competitors, and adjacent industries. Your ecosystem contains early signals about where your components are moving on the evolution curve.

Examples

- Monitoring what early-stage startups are building to spot emerging trends before they're obvious
- A quarterly "customer council" where you hear not just what they like, but where they see the market heading

Leading

Be the owner (take responsibility)

Take full responsibility for outcomes—good and bad. This aligns incentives and prevents organizational learned helplessness. Blame-shifting kills accountability and learning.

Examples

- A CEO publicly owning a market failure and sharing what the leadership team should have done differently
- A product lead taking responsibility for a feature rollback and fixing root cause, not blaming QA

Move fast (an imperfect plan executed today is better than a perfect plan executed tomorrow)

Favor speed of execution over perfection. Moving creates learning and market feedback; waiting creates risk of irrelevance and gives competitors first-mover advantage.

Examples

- Shipping a feature with known rough edges to capture a market window that closes in 3 months
- Making strategic bets with 70% information today rather than waiting for 95% certainty (by which time the market moved)

Think big (inspire others, provide direction)

Set ambitious vision and goals that inspire and align teams. The specifics may change, but clear direction prevents local optimization that contradicts strategy.

Examples

- A mission statement like "become the operating system for [industry]" that guides thousands of daily decisions, not a vanity statement
- A 10-year vision that's ambitious enough that it changes how teams prioritize today

Strategy is iterative not linear (fast reactive cycles)

Continuously adapt strategy based on learning, rather than committing to a multi-year plan and executing rigidly. Markets move faster than most planning cycles; waiting kills advantage.

Examples

- Monthly capability reviews adjusting what to build next based on early customer usage data, not executing the 18-month roadmap unchanged
- Abandoning a go-to-market approach mid-quarter when you discover a different buyer persona actually values your product more

Strategy is complex (there will be uncertainty)

Accept that the future is uncertain and complex systems resist prediction. Plan for multiple scenarios and build resilient strategies, don't bet the company on one outcome.

Examples

- War-gaming three scenarios (recession, rapid growth, new entrant) and building contingency plans for each
- Building a supply chain with redundancy for disruptions you can't predict, instead of optimizing for one scenario

Commit to the direction, be adaptive along the path (crossing the river by feeling the stones)

Stay committed to the strategic direction while remaining flexible about the exact path. Waypoints matter; the route to reach them should pivot with new information.

Examples

- Vision: "Become the enterprise standard for [category]." This never changes; GTM strategy (direct, channel, partnerships) pivots quarterly based on what works
- Committed to geographic expansion but changing which markets to enter first as you learn what's achievable

There is no core (everything is transient)

Everything—products, processes, competitive advantages—is temporary. Avoid becoming attached to existing success. This is the hardest doctrine to live; success breeds the inertia that kills strategy.

Examples

- A SaaS company sunsetting their legacy on-premise product line despite it generating 40% of revenue, before commoditization destroys the entire business
- Regularly asking "if we started today, would we still build this?" and having the courage to say no

Be humble (listen, be selfless, have fortitude)

Listen more than you speak; solve the problem, not your ego; persist through difficulty without losing perspective or becoming brittle. Leaders who've lost humility crash.

Examples

- A leader asking teams what they're seeing in customer data instead of entering the room with conclusions
- Admitting "we were wrong, we're changing direction" publicly and adjusting compensation/reviews accordingly

Exploit the landscape

Understand your strategic position via mapping, anticipate change in your components' evolution, and act with conviction at the moment of maximum leverage. Being too early or too late destroys value.

Examples

- A company mapping trends, recognizing when cloud transitions from custom to product, and shifting investment 6 months before competitors realize the change
- Acquiring a complementary company when your category transitions to commodity, before margins collapse and prices inflate

Climatic Patterns

Components	Everything evolves through supply and demand competition	Rates of evolution can vary by ecosystem (e.g. consumer vs Industrial)	Characteristics change as components evolve (Salaman & Storey)	No choice over evolution (Red Queen)
	No single method fits all (e.g. in development or purchasing)	Components can co-evolve (e.g. practice with activity)	Evolution consists of multiple waves of diffusion with many chasms	
Financial	Higher order systems create new sources of value	Efficiency does not mean a reduced spend (Jevan's Paradox)	Capital flows to new areas of value	Creative Destruction (Joseph Schumpeter)
	Future value is inversely proportional to the certainty we have over it	Evolution to higher order systems results in increasing local order and energy consumption		
Speed	Efficiency enables innovation	Evolution of communication mechanisms can increase the speed of evolution overall and the diffusion of a single example of change	Increased stability of lower order systems increases agility & speed of re-combination	Change is not always linear (discontinuous and exponential change exists)
	Shifts from product to utility tend to demonstrate a punctuated equilibrium			
Inertia	Success breeds inertia	Inertia can kill an organization	Inertia increases the more successful of the past model is	
Competitors	Competitors actions will change the game	Most competitors have poor situational awareness		
Prediction	Not everything is random (p[what] vs p[when])	Economy has cycles (peace, war and wonder)	Two different forms of disruption (predictable vs nonpredictable)	A "war" (point of industrialization) causes organizations to evolve
	You cannot measure Evolution over time or adoption, you need to embrace uncertainty	Evolution consists of multiple diffusion curves	The less evolved something is then the more uncertain it becomes	

Climatic Patterns Reference

Components

Everything evolves through supply and demand competition

Components evolve because market pressure rewards better, cheaper, faster, and more reliable forms. What starts as novel custom work is forced toward productization and eventually utility.

Examples

- Cloud computing moved from custom hosting projects to standardized IaaS because buyers prioritized speed and cost.
- Payment APIs became commodities as demand shifted from bespoke integrations to low-friction, developer-ready services.

Rates of evolution can vary by ecosystem (e.g. consumer vs Industrial)

Evolution speed is context-dependent. Consumer markets evolve rapidly under short feedback loops, while industrial sectors evolve slower due to regulation, safety, and capital intensity.

Examples

- Consumer messaging apps changed feature paradigms in months; industrial ERP replacement cycles often take years.
- EV software updates ship weekly, while aerospace avionics changes may require multi-year certification.

Characteristics change as components evolve (Salaman & Storey)

As components evolve, buying criteria shift from “Can it work?” to “How reliable, interoperable, and cheap is it?” Uncertainty drops while operational expectations rise.

Examples

- Early AI models were judged on novelty; now buyers compare uptime, latency, compliance, and cost-per-token.
- Databases shifted from feature differentiation to expectations of backup, failover, observability, and managed operations.

No choice over evolution (Red Queen)

You cannot opt out of evolution. Standing still means relative decline because competitors and ecosystems continue moving.

Examples

- Retailers that delayed e-commerce lost share even with strong physical footprints.
- Enterprises that ignored cloud-native patterns faced higher delivery cost and slower release velocity than peers.

No single method fits all (e.g. in development or purchasing)

Methods must match stage of evolution. Genesis requires experimentation, Product needs product management discipline, and Commodity favors optimization and efficiency.

Examples

- Running six-sigma-style controls on a discovery prototype kills learning speed.
- Using open-ended R&D procurement for commodity laptops inflates cost and procurement cycle time.

Components can co-evolve (e.g. practice with activity)

Components rarely evolve alone. Practices, tooling, organization design, and doctrine often evolve together with the technology.

Examples

- CI/CD maturity co-evolved with microservices adoption; teams changed release governance and platform tooling together.
- Data products evolved with data governance practices; one without the other led to low trust and poor adoption.

Evolution consists of multiple waves of diffusion with many chasms

Adoption is not one smooth curve. Different buyer groups adopt at different times, and transitions between groups create chasms where many offerings stall.

Examples

- AR/VR saw early enthusiasm, then a mainstream adoption gap before practical enterprise use cases revived momentum.
- Electric vehicles crossed from early adopters to mainstream only after charging networks, incentives, and model variety matured.

Financial

Higher order systems create new sources of value

When lower-order components commoditize, value shifts upward into orchestration, experience, intelligence, and new business models.

Examples

- Commodity cloud infrastructure enabled new value in SaaS workflows and AI-assisted productivity.
- Cheap GPS created massive value in ride-hailing, logistics optimization, and location-based services.

Efficiency does not mean a reduced spend (Jevan’s Paradox)

Efficiency lowers unit cost, which often increases total consumption. Organizations can spend more overall while becoming more efficient per unit.

Examples

- Cheaper cloud compute led teams to run more workloads and experiments, increasing total cloud spend.
- Better ad targeting lowered cost per conversion but increased total ad budget due to higher measurable ROI.

Capital flows to new areas of value

As value migrates, capital follows. Funding leaves saturated layers and concentrates where differentiation and margin still exist.

Examples

- Venture capital shifted from basic infrastructure tools toward AI application layers and vertical copilots.
- Telco margins compressed in connectivity while capital moved into platforms, content, and data services.

Creative Destruction (Joseph Schumpeter)

New structures replace old ones, destroying incumbent advantages while creating new winners. Defense of legacy often delays but rarely stops this cycle.

Examples

- Streaming displaced DVD rental and linear media distribution models.
- Digital banking products eroded fee-heavy branch-centric banking economics.

Future value is inversely proportional to the certainty we have over it

The highest upside usually sits where certainty is lowest. As certainty rises, value capture potential often compresses due to competition.

Examples

- Early investment in mobile ecosystems created outsized upside; late entrants competed on margin.
- Foundational bets in generative AI carried high uncertainty but produced disproportionate strategic options.

Evolution to higher order systems results in increasing local order and energy consumption

Higher-order systems create more local coordination and capability, but this usually requires greater energy, compute, and operational overhead.

Examples

- AI inference at scale improved automation quality while sharply increasing data center power demand.
- Real-time logistics optimization reduced delivery waste but increased compute, telemetry, and platform complexity.

Speed

Efficiency enables innovation

When foundational activities become efficient, resources are freed for experimentation in higher-value domains.

Examples

- Managed cloud services reduced infrastructure toil, freeing teams to build customer-facing differentiation.
- Standardized payments infrastructure let startups focus on product innovation instead of compliance plumbing.

Evolution of communication mechanisms can increase the speed of evolution overall and the diffusion of a single example of change

Better communication channels accelerate both learning and imitation, compressing the time from invention to widespread adoption.

Examples

- Open-source communities propagated Kubernetes operational patterns globally within a few years.
- Social distribution and developer communities turned prompt engineering techniques into mainstream practice rapidly.

Increased stability of lower order systems increases agility & speed of re-combination

Stable utilities and standards make recombination easier. Teams can assemble new offerings quickly when building blocks are dependable.

Examples

- Stable cloud primitives enabled rapid assembly of modern SaaS stacks.
- Mature API ecosystems allowed fintechs to launch new financial products without building core banking from scratch.

Change is not always linear (discontinuous and exponential change exists)

Evolution often includes sudden jumps, phase shifts, and exponential effects. Linear planning misses regime-change events.

Examples

- Generative AI capabilities improved step-change style, invalidating roadmaps based on incremental assumptions.
- Smartphone adoption created nonlinear shifts in media, retail, and mobility behavior.

Shifts from product to utility tend to demonstrate a punctuated equilibrium

Long periods of incremental change are often interrupted by rapid standardization and utility transition once economic and technical thresholds are crossed.

Examples

- Compute transitioned from on-prem product procurement to cloud utility in a compressed period after hyperscale maturity.
- Video conferencing moved from enterprise product deployments to utility-like default service after widespread remote work demand.

Inertia

Success breeds inertia

Past success creates confidence in current models, incentives, and structures. That confidence often delays adaptation to new landscape realities.

Examples

- A profitable enterprise software firm ignored usage-based pricing too long because perpetual licenses were still strong.
- A market-leading handset maker underestimated touchscreen disruption due to historical success with hardware keyboards.

Inertia can kill an organization

When adaptation lags behind landscape shifts, inertia converts from operational drag to existential risk.

Examples

- A retailer with strong store economics failed to build digital logistics and lost relevance.
- A media company optimized legacy ad inventory while audience attention moved to digital platforms.

Inertia increases the more successful of the past model is

The stronger yesterday's model, the harder it is to abandon. Economic dependence and cultural identity deepen resistance.

Examples

- A company with high-margin legacy hardware delayed subscription transition despite clear market signals.
- A consulting firm built on billable hours resisted productized services even as clients demanded outcome-based pricing.

Competitors

Competitors actions will change the game

Competitive moves alter customer expectations, cost structures, and strategic options. Your map must update as rivals reshape the terrain.

Examples

- A rival bundles AI features at no extra cost, forcing the category baseline upward overnight.
- A new entrant open-sources a core tool, collapsing differentiation based on proprietary access.

Most competitors have poor situational awareness

Many firms optimize for current assumptions and miss landscape shifts. Better situational awareness becomes a strategic weapon.

Examples

- Incumbents dismissed cloud-native challengers as niche until procurement behavior had already shifted.
- Established vendors focused on feature parity while buyers moved attention to ecosystem interoperability.

Prediction

Not everything is random (p[what] vs p[when])

You may predict what is likely to evolve, but not exactly when transitions will occur. Strategic advantage comes from preparing options before timing is obvious.

Examples

- It was predictable that compute would utility-shift; timing varied by industry readiness and regulation.
- It was clear AI assistants would emerge; exact enterprise adoption timing depended on governance and trust barriers.

Economy has cycles (peace, war and wonder)

Economic and industrial activity cycles through exploration, competition, and consolidation phases, each requiring different strategic posture.

Examples

- In “wonder,” firms fund exploratory bets; in “war,” they emphasize scale, efficiency, and survival.
- During downturns, commodity optimization rises; during expansion, adjacent innovation investment increases.

Two different forms of disruption (predictable vs nonpredictable)

Some disruptions are foreseeable from evolution patterns; others are shock events. Strategy needs both directional bets and resilience buffers.

Examples

- Predictable: on-prem software moving to SaaS.
- Nonpredictable: sudden geopolitical sanctions reshaping supply chains in weeks.

A “war” (point of industrialization) causes organizations to evolve

Industrialization points intensify competition, forcing organizations to redesign structure, doctrine, and operating model.

Examples

- As cloud became industrialized, enterprises created platform teams and FinOps to stay competitive.
- As AI tooling industrialized, firms shifted from isolated pilots to centralized model governance and product integration.

You cannot measure Evolution over time or adoption, you need to embrace uncertainty

Time-based tracking alone is misleading. Evolution is nonlinear and context-specific; strategy must combine signals, scenarios, and adaptive decisions.

Examples

- Two markets adopted the same technology at different speeds due to regulation and switching costs.
- A company used scenario triggers (cost, reliability, regulation) instead of calendar milestones for migration decisions.

Evolution consists of multiple diffusion curves

Each adjacent market, segment, and geography may have its own diffusion curve. “Adopted” in one segment does not mean “adopted” everywhere.

Examples

- AI coding tools diffused quickly in startups but slower in regulated enterprises.
- Mobile payments matured rapidly in some countries while remaining early-stage in others.

The less evolved something is then the more uncertain it becomes

Early-stage components carry high uncertainty in value, adoption, and viability. This demands experimentation, small bets, and option thinking.

Examples

- An early quantum software initiative was managed as exploratory options, not committed roadmap delivery.
- A new market-facing AI feature was tested with narrow cohorts before broad rollout and organizational commitment.

Gameplays

User Perception	Education	Bundling	Creating artificial need	Confusion of choice
	Brand and marketing	Fear, uncertainty and doubt	Artificial competition	Lobbying / counterplay
Accelerators	Market enablement	Open approaches	Exploiting network effects	Co-operation
	Industrial policy			
De-accelerators	Exploiting constraint	IPR	Creating constraints	Limitation of competition
Dealing with toxicity	Pig in a poke	Disposal of liability	Sweat and dump	Refactoring
Market	Differentiation	Pricing policy	Buyer / supplier power	Harvesting
	Standards game	Last man standing	Signal distortion	Trading
Defensive	Threat acquisition	Raising barriers to entry	Procrastination	Defensive regulation
	Limitation of competition	Managing inertia		
Attacking	Directed investment	Experimentation	Center of gravity	Undermining barriers to entry
	Fool’s mate	Press release process		
Ecosystem	Alliances	Co-creation	Sensing engine (ILC)	Tower and moat
	Two factor markets	Co-opting and intercession	Embrace and extend	Channel conflicts and
Competitor	Ambush	Fragmentation play	Reinforcing competitor inertia	Sapping
	Misdirection	Restriction of movement	Talent raid	
Positional	Land grab	First mover	Fast follower	Weak signal / horizon
Poison	Licensing play	Insertion	Designed to fail	

Gameplays Reference

User Perception

Education

Increase user understanding of a new capability so they can evaluate it correctly. Education reduces fear, shortens adoption cycles, and shifts buying criteria from myth to evidence.

Examples

- A platform vendor launches a free “Cloud 101 for CFOs” course that explains unit economics, moving deals from stalled to approved.
- A cybersecurity startup publishes incident simulations showing how its product contains breaches, helping buyers justify budget.

Bundling

Package multiple capabilities into one offer to raise perceived value, reduce comparison friction, and lock in demand around a broader proposition.

Examples

- A SaaS firm bundles analytics, support, and compliance reporting into one enterprise tier, making single-feature competitors look incomplete.
- A telecom bundles mobile, broadband, and streaming so churn drops because leaving means replacing several services at once.

Creating artificial need

Shape perception so users believe they require a capability before genuine demand exists. This can accelerate adoption but risks backlash if value is weak.

Examples

- A device maker promotes annual camera upgrades as “essential for creators,” despite marginal real-world differences.
- A B2B vendor frames a minor dashboard feature as a board-level requirement, driving upsell before proven necessity.

Confusion of choice

Overwhelm buyers with options, tiers, or terminology so rational comparison becomes hard, often nudging them toward default or premium choices.

Examples

- A cloud provider introduces many near-identical SKUs with subtle limits, causing buyers to overbuy to avoid risk.
- A software vendor uses complex licensing matrices so procurement chooses the “safe” high-priced bundle.

Brand and marketing

Use reputation, narrative, and emotional trust to influence purchase decisions when technical differences are small or hard to evaluate.

Examples

- Two similar CRM tools compete; the one with stronger executive brand wins enterprise deals despite feature parity.
- A startup with strong thought leadership secures pilot access faster because customers trust its strategic narrative.

Fear, uncertainty and doubt

Amplify perceived risks of alternatives to slow competitor adoption or preserve incumbent advantage. Often used in late-stage defensive moves.

Examples

- An incumbent warns that moving to open-source will trigger “compliance exposure,” without evidence, delaying migration.
- A vendor implies integration failures if customers switch platforms, extending renewal cycles through anxiety.

Artificial competition

Create the appearance of a competitive market while controlling key options, channels, or terms behind the scenes.

Examples

- A company operates multiple “independent” brands with near-identical products, dominating comparison sites.
- A marketplace favors its own private-label variants while presenting them as neutral alternatives.

Lobbying / counterplay

Influence policy, standards, or regulation to protect position or blunt rival strategy; counterplay is the response to such influence.

Examples

- Incumbent payment processors lobby for strict licensing rules that raise startup entry costs.
- A challenger forms an industry coalition to oppose restrictive regulation and push interoperable standards.

Accelerators

Market enablement

Remove adoption barriers (skills, financing, integration, trust) so the market can scale faster.

Examples

- A robotics vendor offers training + financing + implementation partners, turning hesitant pilots into production rollouts.
- A cloud database startup provides migration tooling and certified partners, accelerating enterprise adoption.

Open approaches

Use open standards, open source, or transparent interfaces to expand ecosystem participation and speed diffusion.

Examples

- A company open-sources its SDK, attracting contributors and expanding integrations faster than closed competitors.
- An API provider adopts open schemas so customers can switch tools without re-architecting, increasing market trust.

Exploiting network effects

Design the system so each new user increases value for others, creating compounding growth and defensive strength.

Examples

- A marketplace reaches critical mass; more sellers attract more buyers, which attracts more sellers.
- A collaboration tool gains value as more teams join, because shared documents and workflows become cross-company assets.

Co-operation

Collaborate with peers, even competitors, on non-differentiating layers to expand the total market and reduce duplicated cost.

Examples

- Competing banks co-build a fraud data-sharing utility while competing on customer experience.
- SaaS vendors jointly define identity standards so enterprise onboarding becomes easier for all.

Industrial policy

Leverage government policy, incentives, or procurement direction to accelerate industry transition and local capability buildup.

Examples

- Tax credits for renewable energy trigger rapid deployment and supplier ecosystem growth.
- Public-sector digital procurement mandates cloud-first architectures, accelerating vendor modernization.

De-accelerators

Exploiting constraint

Use bottlenecks (skills, access, regulation, infrastructure) to slow a rival market's growth or control migration pace.

Examples

- A dominant chip supplier prioritizes its own platform partners, slowing competitor hardware releases.
- A vendor restricts API rate limits for third parties, reducing rival product performance.

IPR

Use patents, copyrights, and licensing rights to defend advantage, extract rents, or block imitation.

Examples

- A biotech firm uses patent walls to delay generic competition and preserve premium pricing.
- A software firm enforces restrictive licensing to prevent cloud providers from commoditizing its core engine.

Creating constraints

Intentionally design technical, commercial, or contractual limits that narrow user options and slow substitution.

Examples

- A SaaS contract imposes punitive data-export fees, discouraging platform switching.
- A vendor ties advanced features to proprietary hardware modules, making migration costly.

Limitation of competition

Reduce competitive intensity through exclusivity, control of channels, or structural barriers that restrict challenger reach.

Examples

- A supplier signs long exclusive agreements with major distributors, blocking new entrants.
- A dominant app ecosystem enforces rules that limit alternative payment providers.

Dealing with toxicity

Pig in a poke

Sell or acquire something opaque where real quality is hidden, often shifting uncertainty and downside to the other party.

Examples

- A company sells a “complete AI platform” with no model transparency; buyer discovers poor accuracy after integration.
- An acquirer buys a startup on growth metrics alone, then finds major revenue was discount-driven and non-recurring.

Disposal of liability

Transfer risk, cost, or legal burden to another party through structure, contracts, or spin-outs.

Examples

- A company sells a “complete AI platform” with no model transparency; buyer discovers poor accuracy after integration.
- An acquirer buys a startup on growth metrics alone, then finds major revenue was discount-driven and non-recurring.

Pig in a poke

Sell or acquire something opaque where real quality is hidden, often shifting uncertainty and downside to the other party.

Examples

- A manufacturer outsources hazardous waste operations to isolate environmental liability.
- A conglomerate spins off a debt-heavy unit so core valuation improves.

Sweat and dump

Extract short-term value aggressively (cost cutting, underinvestment), then exit before long-term deterioration is visible.

Examples

- A private owner slashes maintenance and training, boosts short-term EBITDA, then sells at a multiple.
- A digital publisher floods ad inventory and cuts editorial quality before divesting the asset.

Refactoring

Reorganize assets, processes, or architecture to isolate toxicity and restore strategic agility without full replacement.

Examples

- A bank separates legacy core systems behind APIs, allowing modern channels to evolve independently.
- A company splits a politically toxic division into a standalone unit with clear accountability and economics.

Market

Differentiation

Create meaningful distinction in user value where commoditization pressure is rising, so price is not the only lever.

Examples

- A commodity cloud storage provider differentiates with industry-specific compliance workflows.
- A coffee chain differentiates through experience and convenience, not bean chemistry.

Pricing policy

Use price structure strategically (subscription, usage, freemium, bundles) to shape adoption, margin, and competitive behavior.

Examples

- A SaaS firm shifts from seat-based to usage-based pricing to align with customer value and improve expansion revenue.
- A challenger uses freemium entry + paid collaboration features to undercut incumbent acquisition cost.

Buyer / supplier power

Exploit or rebalance bargaining power in the value chain to improve terms, resilience, or market control.

Examples

- A retailer launches private-label products to reduce dependence on branded suppliers.
- A chipmaker with scarce capacity negotiates prepaid contracts and long-term commitments from buyers.

Harvesting

Maximize cash from mature offerings by limiting reinvestment while demand still exists.

Examples

- A software vendor places a legacy on-prem suite in maintenance mode, optimizing profit while funding cloud products.
- A media company monetizes a declining print segment through premium subscriptions before exit.

Standards game

Shape standards to favor your strengths, reduce switching friction for your ecosystem, or weaken proprietary rivals.

Examples

- A consortium pushes an open protocol where one member already has a mature implementation.
- A platform vendor contributes heavily to a standard body to ensure compatibility with its tooling stack.

Last man standing

Endure prolonged margin pressure better than rivals, then capture share and pricing power after consolidation.

Examples

- A logistics player survives a price war through superior balance sheet and automation, then acquires distressed competitors.
- A cloud provider sustains low pricing until smaller hosts exit, then expands higher-margin services.

Signal distortion

Manipulate market signals (metrics, messaging, timing) to influence competitor or investor decisions.

Examples

- A firm announces aggressive roadmap milestones to freeze customer purchasing from rivals.
- A company emphasizes vanity MAU metrics while hiding retention decline, encouraging competitor overreaction.

Trading

Use market timing, arbitrage, or position swaps (assets, contracts, capacity) to capture value from movement and asymmetry.

Examples

- An energy-intensive data operator hedges power contracts during low-price windows, gaining cost advantage.
- A cloud reseller arbitrages regional pricing and reserved capacity to improve gross margin.

Defensive

Threat acquisition

Acquire emerging threats early to neutralize competitive risk or internalize disruptive capability.

Examples

- An incumbent design tool acquires a fast-growing collaborative startup before it moves upmarket.
- A payments giant buys a developer-first API company to prevent ecosystem disintermediation.

Raising barriers to entry

Increase the cost, complexity, or time required for newcomers to compete effectively.

Examples

- A platform secures exclusive data partnerships that are expensive for entrants to replicate.
- A pharma firm expands clinical evidence and distribution depth beyond what startups can fund.

Procrastination

Delay necessary change to preserve short-term economics or internal politics, often disguised as “further analysis.”

Examples

- A telco delays core network modernization for years while competitors launch digital-first plans.
- A software company postpones API modernization, accumulating integration debt that later forces a crisis migration.

Defensive regulation

Use compliance and policy mechanisms to slow challenger entry or protect incumbent operating models.

Examples

- Legacy financial institutions push stricter onboarding requirements that small fintechs struggle to operationalize.
- An industry body introduces costly certification cycles that favor large incumbents.

Limitation of competition

Constrain rival access to customers, channels, or critical dependencies to reduce market contestability.

Examples

- A dominant app store privileges first-party apps in ranking and distribution.
- A vendor uses long lock-in contracts with auto-renewal terms across key enterprise accounts.

Managing inertia

Intentionally address internal resistance (incentives, legacy assets, politics) so adaptation becomes executable.

Examples

- A bank ties leadership bonuses to cloud migration milestones, not only legacy system uptime.
- A manufacturer funds retraining and role redesign before automation rollout to reduce organizational backlash.

Attacking

Directed investment

Concentrate resources on specific strategic choke points rather than spreading capital across many weak bets.

Examples

- A logistics startup invests heavily in one high-volume lane to prove superior economics before expansion.
- A SaaS challenger funds only two integrations that unlock 70% of target enterprise accounts.

Experimentation

Run fast, low-cost probes to discover winning approaches under uncertainty before scaling commitment.

Examples

- A retailer tests three checkout flows on small cohorts and scales the one with best completion rate.
- A B2B startup pilots pricing models in two segments before company-wide rollout.

Center of gravity

Identify and attack the opponent's critical dependency (profit engine, channel, trust anchor) to unbalance their strategy.

Examples

- A challenger targets the incumbent's premium support reputation by offering faster guaranteed response with transparent SLAs.
- A new entrant attacks a competitor's strongest vertical by delivering a radically simpler workflow for that niche.

Undermining barriers to entry

Remove structural obstacles (cost, skill, tooling, access) so more players can participate and incumbent control weakens.

Examples

- A platform releases no-code builders that let small merchants launch without developers.
- An open API standard eliminates proprietary integration gates used by incumbents.

Fool's mate

Create a rapid sequence of moves that exploits predictable responses, producing outsized advantage before rivals adjust.

Examples

- A startup launches free migration tools, then low-risk pilots, then usage pricing—competitors react too late and lose accounts.
- A new cloud service offers compatibility + credits + partner incentives in one quarter, collapsing switch friction quickly.

Press release process

Work backwards from a future customer announcement to force clarity on value, target user, and differentiators before building.

Examples

- A product team writes a launch press release for "1-hour onboarding" and kills features that do not support that promise.
- An enterprise unit drafts FAQ + metrics for a future launch, revealing missing capabilities early.

Ecosystem

Alliances

Form strategic partnerships to combine complementary strengths, extend reach, and shape market structure.

Examples

- A cloud provider and consultancy create a joint migration program to accelerate enterprise adoption.
- A health-tech company partners with insurers to reduce customer acquisition friction.

Co-creation

Build solutions with customers or partners directly so offerings match real operational context and gain early advocates.

Examples

- An industrial IoT vendor co-develops a predictive maintenance module with one flagship factory.
- A B2B software team embeds with a design partner for 8 weeks to shape workflow features.

Sensing engine (ILC)

Use the ecosystem as an intelligence loop to detect weak signals, emerging needs, and landscape shifts earlier than rivals.

Examples

- A platform tracks partner API requests to spot new demand before formal RFPs appear.
- Quarterly councils with customers, startups, and integrators surface upcoming compliance changes early.

Tower and moat

Build a valuable core (tower) and reinforce defensibility (moat) through switching costs, network effects, data, or ecosystem dependency.

Examples

- A workflow platform becomes core operating software, then deepens moat via embedded automations and proprietary usage data.
- A marketplace builds trust systems and logistics infrastructure that are expensive for rivals to replicate.

Two factor markets

Operate two-sided markets where value depends on balancing participation and incentives across both sides.

Examples

- A ride-hailing platform subsidizes riders in low-liquidity periods while guaranteeing driver earnings to maintain supply.
- A B2B platform balances buyer procurement tools with supplier analytics so both sides stay active.

Co-opting and intercession

Absorb or redirect external movements (communities, standards, partners) so they align with your strategic path.

Examples

- A vendor adopts a popular open-source extension model, then becomes the preferred enterprise distribution.
- An incumbent invites critics into a standards committee and channels demands into compatible governance.

Embrace and extend

Support widely adopted standards to gain entry, then add proprietary extensions that pull users deeper into your ecosystem.

Examples

- A platform fully supports an open data format, then adds premium workflow features available only in its cloud.
- A productivity suite imports competitor files perfectly, then promotes collaboration features that lock team usage.

Channel conflicts and disintermediation

Manage tension when direct digital channels bypass existing intermediaries, redistributing margin and control.

Examples

- A manufacturer launches direct e-commerce, triggering pushback from distributors over pricing and territory.
- A SaaS vendor shifts from reseller-led to product-led growth, forcing partner compensation redesign.

Competitor

Ambush

Launch a targeted move where incumbents are inattentive or structurally weak, gaining disproportionate ground quickly.

Examples

- A niche SaaS player attacks one underserved compliance workflow and wins flagship enterprise logos.
- A low-cost carrier enters a high-fare route with limited slots, forcing incumbents into defensive pricing.

Fragmentation play

Break a rival's integrated offer into separable components and compete selectively where they are weakest.

Examples

- A fintech unbundles banking into payments, lending, and treasury APIs, eroding full-suite incumbents.
- A software challenger replaces only the analytics module in an ERP stack, then expands footprint.

Reinforcing competitor inertia

Encourage rivals to stay committed to outdated models so they delay adaptation while you reposition.

Examples

- A disruptor praises incumbent “enterprise rigor,” while quietly winning SMB adoption with faster onboarding.
- A challenger lets rivals keep optimizing on-prem margins while it captures cloud-native customers.

Sapping

Drain competitor resources through sustained pressure (price, support burden, feature demands) until strategic flexibility collapses.

Examples

- A company repeatedly bids low on key accounts to force a cash-constrained rival into margin erosion.
- A platform floods a competitor’s roadmap with integration requirements from shared enterprise customers.

Misdirection

Draw competitor attention toward non-critical areas while your real strategic move develops elsewhere.

Examples

- A startup publicizes minor UI experiments while investing heavily in backend cost advantage.
- A rival signals expansion into one geography, causing incumbents to react, then launches in a different high-margin segment.

Restriction of movement

Limit a competitor’s strategic options by controlling dependencies, standards, or channels.

Examples

- A cloud provider secures exclusive AI model hosting rights, reducing alternatives for rivals.
- A marketplace adjusts API policies so third-party tools cannot replicate core workflow capabilities.

Talent raid

Recruit key people from competitors to gain domain expertise, execution speed, and inside understanding of their constraints.

Examples

- A challenger hires a rival’s platform engineering leaders to accelerate reliability improvements.
- A fintech recruits experienced compliance specialists from incumbents, cutting go-to-market delay.

Positional

Land grab

Capture scarce strategic positions early (customers, channels, data, contracts) before market structure settles.

Examples

- A SaaS startup signs multi-year contracts with the top 20 enterprise logos in a vertical.
- A mobility platform secures airport and city permits early, limiting room for later entrants.

First mover

Enter early to shape user expectations, standards, and ecosystem relationships, accepting higher uncertainty.

Examples

- A company launches the first practical AI assistant for a niche profession and becomes default tooling.
- An early API provider defines integration patterns that later competitors must follow.

Fast follower

Let pioneers absorb discovery cost, then execute quickly with improved economics, distribution, or usability.

Examples

- A firm enters after category validation, offering simpler pricing and better onboarding to win mainstream users.
- A hardware maker waits for component costs to fall, then scales aggressively with superior supply chain efficiency.

Weak signal / horizon

Detect and act on early indicators of future shifts before they are visible in mainstream metrics.

Examples

- Rising developer interest in a new protocol prompts early tooling investment before enterprise demand spikes.
- A consumer brand monitors small cohort behavior shifts and prepositions inventory months ahead of trend adoption.

Poison

Licensing play

Use licensing structures to constrain competitors, extract value, or force ecosystem dependence on your terms.

Examples

- A software owner changes from permissive to restrictive licensing to block cloud resellers.
- A platform offers low entry license fees but expensive scale clauses, capturing upside as customers grow.

Insertion

Place your component deep inside another value chain so removal becomes costly, then expand influence from that foothold.

Examples

- A monitoring vendor embeds agents in core workloads, then expands to security and incident response products.
- A payments processor integrates into checkout stacks and later introduces financing and fraud tools.

Designed to fail

Promote an option likely to underperform so stakeholders reject that direction and accept your preferred alternative.

Examples

- A team pilots cloud migration with impossible constraints, then cites failure to justify staying on legacy systems.
- A vendor launches a deliberately weak low-cost tier so customers quickly “prove” premium is necessary.

Signals of

Higher levels of situational awareness	Indicator of …	Lower levels of situational awareness
Visual (maps)	Navigation	Verbal (story telling)
Action & reflection	Learning	Secrets of success
Position and movement	Strategy	Magic thinking (SWOTs, 2x2s)
Where before why	Order	Tyranny of action (how, what and when)
Detailed	Landscape	Vague
Context specific & universal	Patterns	Universal memes
Aptitude and attitude	Organization	Functional or business silos
Thinks small	Granularity	Thinks big
Mixed (agile + lean + six sigma)	Methods	Single (all agile, etc.)
Known	Duplication and bias	Unable to quantify
Embraced	Challenge	Defended against
Essential	Transparency	Resisted
Common	Language	Tribal factions
User needs	Anchor	Own needs